1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA
DEPARTAMENTO DE ENGENHARIA ELECTROTÉCNICA E DE COMPUTADORES

Robotics (02000537)

Trabalho Laboratorial 0 — Parte 2: Ciclo de Vida e Ambiente de Desenvolvimento

Labwork 0 — Part 2: Development Lifecycle and Development Environment

Jorge Batista · **João Quintas** · **Carlos Azevedo**

1.º Semestre / 1st Semester, 2026/2027
Grupos de 2 estudantes / Groups of 2 students · Semana / Week 3

[PT] — Versão Portuguesa

Onde é que isto se encaixa

Na **parte 1** comandaram o robô com as ferramentas que o próprio robô fornece. Esse é o *nível básico*, e não escala: não se pode versionar um script escrito numa página web, nem testar uma trajetória que não tem outro sítio para correr senão o braço real.
Este documento faz duas coisas. Primeiro introduz o **ciclo de vida de desenvolvimento** usado em sistemas robóticos — como se gerem requisitos, conceção, testes e configuração — que é a disciplina contra a qual todos os trabalhos e o relatório do Desafio Final são avaliados. Depois instala os dois níveis de trabalho restantes:

Nível	Com o que trabalham	Usado para
Básico (parte 1)	Studio, Live Control, o IDE Python do robô	primeiro contacto, verificações rápidas
Intermédio	o vosso computador: ambiente virtual, projeto, IDE, o SDK	**LW1–LW4 e o Desafio Final**
Avançado	ROS 2 Jazzy + MoveIt 2 + Gazebo	simulação, física, a extensão com visão

O Desafio Final é distribuído nesta sessão, como `Robotics_2627_Challenge`. Leiam-no a par da Parte A que se segue: é um exemplo trabalhado de tudo o que este documento ensina sobre requisitos e verificação, e é o objetivo para que os quatro trabalhos restantes constroem. É validado depois das férias de Natal, em data acordada com a turma.
Avaliação. O LW0 vale 0.4 valores, entregue com o LW1 e o LW2 na **Entrega 1 (01/11/2026)** e avaliado no **Checkpoint 1** (semana 8): 75% de entrega, 25% de acompanhamento em aula, mais uma defesa oral pontuada individualmente.

A. O Ciclo de Vida de Desenvolvimento para Sistemas Robóticos

Um sistema robótico falha de formas que o software puro não falha. Tem massa e quantidade de movimento, sensores que mentem, um controlador com opiniões próprias e uma pessoa ao alcance. A

disciplina de ciclo de vida usada em engenharia relacionada com segurança existe para tornar essas falhas visíveis *antes* de acontecerem, e a baixo custo. Esta unidade curricular adota daqui em diante uma versão leve dessa disciplina, e avalia-a explicitamente no Desafio Final, cujo relatório é uma especificação em modelo em V.

A.1 Modelos de ciclo de vida, e porque a robótica se apoia no V

- **Cascata**: requisitos, conceção, implementação, teste, em sequência. Funciona quando nada se aprende pelo caminho; em robótica aprende-se algo em cada camada.
- **Modelo em V**: cada nível de definição no ramo esquerdo é emparelhado com um nível de verificação no ramo direito, e *o teste é planeado antes de o código ser escrito.* É a forma padrão em sistemas automóveis, médicos e robóticos, porque testar com hardware é caro e perigoso, logo tem de ser concebido e não improvisado.
- **Iterativo / incremental**: ciclos curtos, cada um entregando algo que funciona e tem uma interface estável. Esta unidade curricular é exatamente isso: cada trabalho é um incremento com uma **interface congelada** — `fk_lite6`, `ik_lite6`, `stream`, `slide_piece`, `pick_and_place` — consumida pelo seguinte.

A unidade curricular corre **incrementos dentro de um V**: sobem o ramo esquerdo trabalho a trabalho, e o Desafio Final percorre o ramo direito de volta para cima.

Fase do ciclo de vida	Na célula do Lite 6	Onde
Conceito de operações	a tarefa, a célula, os atores, as restrições (alcance, carga, curso)	Desafio
Requisitos	funcionais / desempenho / interface / segurança, cada um com um teste	todas as folhas
Arquitetura	módulos com interfaces congeladas e um fluxo de dados	LW1–LW4
Conceção detalhada	tabela DH, ramos e política de IK, lei de amortecimento, geometria do empurrão, junções	LW1–LW4
Implementação	módulos Python, `git`, `README`, ficheiro de configuração	todos
Verificação unitária	ida-e-volta da cinemática, Jacobiano analítico vs. numérico, verificações de exequibilidade	LW1–LW4
Verificação de integração	um deslizamento e uma captura com as vizinhas ocupadas	LW4
Verificação de sistema	as campanhas de 20 operações, falhas induzidas, tempos	Desafio
Validação	a sessão ao vivo em que o sistema é confrontado com a sua própria especificação	Desafio
Operação e manutenção	registos de cada movimento e código de retorno, análise *a posteriori*	Desafio

A.2 Requisitos que se podem testar

Um requisito é uma frase com *deve*, um sujeito e um resultado mensurável. Quatro tipos: **funcionais** (o que o sistema faz), **de desempenho** (quão bem, com métrica, limiar e alvo), **de interface** (formatos, unidades, protocolos — milímetros e graus no SDK, radianos no ROS 2, uma fonte real de erros) e **de segurança** (o que nunca pode acontecer).

Não é um requisito	É um requisito
O robô deve ser exato.	O sistema deve colocar uma peça a menos de 5 mm (limiar) / 2 mm (alvo) do centro da casa de destino, medido em 20 colocações.
O robô deve ser seguro.	O sistema deve abortar e recuar para `home` em menos de 15 s depois de detetar que a manipulabilidade desceu abaixo do limiar configurado, verificado por três testes de falha induzida.
O empurrão deve ser suave.	A altura da ferramenta deve variar menos de 3 mm pico-a-pico durante um deslizamento, reconstruída a partir dos ângulos de junta registados.

A **rastreabilidade** é o fio que mantém o V unido: requisito → elemento de conceção → teste → resultado medido. Um requisito sem teste é um desejo; um teste que não rastreia para nada é esforço perdido. Todas as folhas desta unidade curricular enunciam os seus requisitos como Rn e os de desempenho como Pn com limiar e alvo, e pedem valores medidos. O documento do Desafio Final já contém um exemplo trabalhado de toda a estrutura: dez requisitos funcionais, dez de desempenho, um protocolo de quinze testes, uma **matriz de cobertura** que mostra que nenhum requisito fica por testar, e uma tabela de rastreabilidade. **Leiam-no a par desta secção** — é o formato sobre o qual serão avaliados, e a sua sessão de validação é pontuada exatamente pela parcela dessas duas listas que conseguirem completar.

A.3 Estratégia de verificação: a escada de simulação

1. **Testes unitários, sem robô.** Python puro: idas-e-voltas de cinemática direta, Jacobianos por diferenças finitas, verificações de exequibilidade. Segundos a correr; corram-nos a cada alteração.
2. **Simulação em malha.** Dois simuladores diferentes, com propósitos diferentes (§C.4). Concebam o código de modo a que trocar entre simulador e robô seja **um parâmetro**, nunca uma edição.
3. **Hardware em malha.** Modo reduzido, aprovação do docente, um membro na paragem de emergência. Tudo medido, nada afirmado.
4. **Regressão.** Quando o LW2 alterar algo de que o LW1 dependia, os testes do LW1 continuam a correr. Mantenham-nos; são a vossa rede de segurança em dezembro.

Regra de reporte para toda a unidade curricular: um requisito de desempenho **não cumprido**, com o valor medido e uma análise do porquê, vale mais do que um dado como cumprido sem evidência.

A.4 Segurança dentro do ciclo de vida

A segurança projeta-se, não se espera. Nesta célula os perigos são concretos: entalamento na garra (6 N é pouco, os dedos são menos), impacto de um braço de 7.2 kg a até 500 mm/s, movimento inesperado quando se limpa um erro, e uma rede partilhada em que vários portáteis podem comandar um controlador. A hierarquia de redução de risco aplicada aqui:

Nível	Neste laboratório
Conceção intrinsecamente segura	limites de velocidade do modo reduzido, fronteira do TCP a envolver a bancada, carga baixa
Medidas técnicas de proteção	paragem de emergência, deteção de colisão, o aborto por singularidade que escrevem no LW3
Procedimentos	aprovação do docente, simulador antes de hardware, um membro na paragem
Informação para utilização	parte 1 §3, a declaração de segurança que assinaram

Os requisitos de segurança são requisitos como os outros, com testes como os outros. Só para orientação, e não examinável: a ISO/IEC/IEEE 15288 e a 12207 descrevem processos de ciclo de vida de sistemas e de software; a IEC 61508 está na origem da disciplina do modelo em V na segurança funcional; a ISO 10218 e a ISO/TS 15066 cobrem a segurança de robôs industriais e colaborativos.

A.5 Gestão de configuração e de alterações

- **Um repositório `git` por grupo.** Etiquetem o que entregam: `delivery-1`, `delivery-2`, `delivery-3`. O que é avaliado é a etiqueta.
- **Parâmetros são dados, não código.** IP do controlador, `ROS_DOMAIN_ID`, *offset* da ferramenta, pose do tabuleiro, limites do modo reduzido, e mais tarde v_{pico}, v_{contacto}, h_{push}, h_{voo} vivem num ficheiro de configuração. O Desafio Final proíbe alterações de código durante a sessão de validação mas permite alterações de parâmetros — regra que só é útil a um grupo que separou as duas coisas em outubro.
- **O `README` reproduz a entrega a partir do zero.** Se não conseguir, a entrega não existe.
- **Registos desde o primeiro dia.** Cada movimento comandado, o código de retorno, o resultado. A fase de manutenção de um robô começa no momento em que ele se move pela primeira vez.

B. Nível Intermédio — O Vosso Computador

O IDE Python web da parte 1 é substituído a partir daqui. Tudo o que entregarem do LW1 em diante é um projeto Python na vossa máquina, sob controlo de versões, executado a partir de um editor a sério.

B.1 Ambiente virtual e pacotes

Um ambiente virtual isola os pacotes desta unidade curricular do resto do sistema. Python $\geq$ 3.10.

```
python3 -m venv ~/venvs/robotica
source ~/venvs/robotica/bin/activate        # Windows: %USERPROFILE%\venvs\robotica\
    Scripts\activate
python -m pip install --upgrade pip
pip install xarm-python-sdk numpy scipy matplotlib pytest pyyaml
pip freeze > requirements.txt               # versionar este ficheiro
```

B.2 O repositório e o SDK

```
git clone https://github.com/xArm-Developer/xArm-Python-SDK.git   # referencia: doc/
    api, example/wrapper
```

Clonem-no **fora** do vosso projeto: é documentação e exemplos, não o vosso código. O pacote que importam vem do `pip`, não deste clone.

B.3 Estrutura do projeto

Usem esta estrutura desde o primeiro dia. É a estrutura que os trabalhos seguintes assumem quando dizem "importem o vosso módulo do LW1".

```
robotica-grupoNN/
  config/robot.yaml          # ip, home, offset da ferramenta, limites, pose do
    tabuleiro, v_pico...
  lite6/
    __init__.py
    session.py               # safe_session() da parte 1
```

```
    kinematics.py            # LW1: fk_lite6, pose_to_T, T_to_pose, workspace_sample
    ik.py                    # LW2: ik_lite6
    velocity.py              # LW3: jacobian, manipulability, rrmc_step, stream
    trajectory.py            # LW4: traj_*, slide_piece, pick_and_place
    game.py                  # Desafio Final: estado do tabuleiro, regras,
   sequenciador de lances
  scripts/                   # um script executavel por experiencia, com o nome do
    teste que corre
  tests/                     # pytest: testes unitarios que nao precisam de robo
  logs/
  requirements.txt
  README.md
```

Why this matters in practice

Porquê um pacote e não uma pasta de scripts. Cada trabalho congela uma interface que o seguinte importa: `from lite6.kinematics import fk_lite6`. Se o LW1 viver em `trabalho1_final_v3.py`, o LW2 não o consegue importar sem copiar, e em dezembro terão quatro cópias de uma função de cinemática direta que discordam entre si. É o ponto de gestão de configuração do §A.5, na sua forma mais concreta.

B.4 O IDE

Usem **VS Code** (com as extensões Python e Pylance) ou **PyCharm**. Qualquer um serve; o que importa é obterem: o interpretador apontado ao vosso ambiente virtual, um depurador com pontos de paragem, e `git` integrado. Definam o interpretador explicitamente — um script que corre no terminal mas não no IDE é quase sempre um interpretador errado.

B.5 Validação do nível intermédio

1. O `pytest` corre e passa com uma suite de testes vazia.
2. O `safe_session.py` da parte 1, movido para `lite6/session.py`, é importado por um script em `scripts/` e liga-se ao robô.
3. Todos os parâmetros de que o script precisa vêm de `config/robot.yaml`; a sequência `192.168` não aparece em lado nenhum do código Python.
4. O repositório tem pelo menos um *commit* e o `README` explica como recriar o ambiente.

C. Nível Avançado — ROS 2 e Gazebo

C.1 Porquê

O SDK fala com o controlador; não simula um robô. O **Gazebo** simula: massa, gravidade, inércia, atrito das juntas, contacto. Um braço no Gazebo cede sob o próprio peso, ultrapassa o alvo e não segue exatamente a trajetória comandada — como o real. O **RViz** é um visualizador, não um simulador: desenha o robô onde a cinemática diz que ele deve estar, e tem sempre razão, porque não está a simular nada. O **MoveIt 2** planeia movimentos sem colisão e dá-vos uma referência independente com que comparar o vosso próprio planeador. Distinguir estes três é metade do valor desta secção.

A pilha-alvo é **ROS 2 Jazzy Jalisco** + MoveIt 2 + Gazebo Harmonic + os pacotes `xarm_ros2` da UFACTORY, sobre **Ubuntu 24.04**. Duas vias: o contentor (recomendada) e a instalação de raiz (opcional).

C.2 Via 1 — Docker (recomendada)

Um contentor dá a todos os grupos o mesmo ambiente, independentemente do que o portátil tem instalado, e tira uma tarde de gestão de pacotes do caminho crítico.

```
# imagem fornecida pela equipa docente
docker pull <registry>/robotica-deec/ros2-jazzy-lite6:2627

# Linux: aplicacoes graficas do contentor no vosso ecra
xhost +local:docker
docker run -it --name ros2lab --net=host \
    -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix \
    -v $HOME/robotica-grupoNN:/work \
    <registry>/robotica-deec/ros2-jazzy-lite6:2627
```

Em **Windows** usem o Docker Desktop com o *backend* WSL2; em **macOS** usem o Docker Desktop e um servidor X (XQuartz) para a interface gráfica, e esperem que o Gazebo seja lento em Apple Silicon, onde a imagem é emulada. Montem a pasta do vosso projeto (`-v` acima) para que o código que editam no IDE no anfitrião seja o código que corre no contentor.

C.3 Via 2 — De raiz (opcional, para os curiosos)

Vale a pena se quiserem perceber o que o contentor contém. Duas variantes: uma **máquina virtual** (VMware Workstation em Windows/Linux, VMware Fusion em macOS; deem-lhe pelo menos 4 núcleos, 8 GB de RAM e 40 GB de disco, e ativem a aceleração 3D ou o Gazebo não desenha) ou uma instalação **nativa** de Ubuntu 24.04, que é a mais rápida de todas.

1. ***Locale.***

```
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

2. **Repositórios.**

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe
```

3. **Fonte apt do ROS 2.** Sigam a secção *Set up sources* da página oficial, `https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html`. É o único passo que muda de vez em quando a montante, pelo que a página oficial é a fonte de verdade. Parem quando `sudo apt update` correr sem erros.

4. **ROS 2, ferramentas, MoveIt, Gazebo.** O ROS 2 Jazzy emparelha com o **Gazebo Harmonic**; não instalem o "Gazebo Classic", que chegou ao fim de vida em janeiro de 2025 e é um programa diferente e incompatível apesar do nome.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ros-jazzy-desktop ros-dev-tools ros-jazzy-moveit \
                    ros-jazzy-ros-gz ros-jazzy-gz-ros2-control
sudo rosdep init && rosdep update
gz sim --version        # esperar 8.x  (Gazebo Sim 8 e o Harmonic)
```

5. ***Shell.*** Acrescentem ao `~/.bashrc` e façam `source`:

```
source /opt/ros/jazzy/setup.bash
export ROS_DOMAIN_ID=<NUMERO DO VOSSO GRUPO>
```

   O `ROS_DOMAIN_ID` não é opcional. Os nós ROS 2 descobrem-se na rede local. Sem ele, na rede do laboratório, o vosso RViz encontra os nós dos colegas, vão ver estados de juntas que não são vossos, e o MoveIt vai planear contra um robô que não existe. Usem o número atribuído ao vosso grupo (Anexo B) e nenhum outro.

6. ***Workspace* e `xarm_ros2`.** A ordem importa: `$ROS_DISTRO` tem de estar já definido, por isso o passo anterior vem primeiro.

```
mkdir -p ~/dev_ws/src && cd ~/dev_ws/src
# NAO omitir --recursive: o SDK C++ que fala com o robo e um submodulo git
git clone https://github.com/xArm-Developer/xarm_ros2.git --recursive -b jazzy
rosdep install --from-paths . --ignore-src --rosdistro jazzy -y
cd ~/dev_ws && colcon build        # 15-30 min; --parallel-workers 2 se faltar RAM
echo "source ~/dev_ws/install/setup.bash" >> ~/.bashrc && source ~/.bashrc
```

Esqueceram-se do `-recursive`? Não é preciso clonar de novo: em `~/dev_ws/src/xarm_ros2` corram `git submodule update -init -recursive` e recompilem.

C.4 Dois simuladores, dois propósitos

	Gazebo (ROS 2)	**Simulador de firmware da UFACTORY**
O que reproduz	a *física*: massa, gravidade, atrito, contacto	o *controlador*: modos, *streaming* de servo, limites, códigos de erro
Comandado por	MoveIt 2 através do `ros2_control`	o xArm Python SDK, exatamente como o braço real
Usar antes de	qualquer movimento planeado pelo MoveIt	qualquer movimento comandado pelo SDK, ou seja, todo o LW3 e LW4

O simulador de firmware é o próprio firmware do controlador da UFACTORY distribuído como contentor. Fala o mesmo protocolo que o braço físico, pelo que **o mesmo script Python corre sem alterações contra ele e contra o robô** — só muda o IP, e é por isso que o IP vive no `config/robot.yaml`. É apenas amd64, e emulado em Apple Silicon.

```
docker pull danielwang123321/uf-ubuntu-docker
docker run -it --name uf_software -p 18333:18333 -p 502:502 -p 503:503 -p 504:504 \
    -p 30000:30000 -p 30001:30001 -p 30002:30002 -p 30003:30003 \
    danielwang123321/uf-ubuntu-docker
# dentro do contentor, arrancar o firmware escolhendo o perfil Lite 6 (argumentos "6
    9");
# ver o artigo de suporte da UFACTORY "How to install UFACTORY Studio in Docker"
# depois: browser -> http://127.0.0.1:18333   Python -> XArmAPI('127.0.0.1',
    check_joint_limit=False)
```

C.5 Testar o nível avançado

Corram-nos **por ordem**; cada um assume que o anterior passou. Abram um terminal novo para cada um salvo indicação em contrário. Registem todos os resultados — é um entregável.

Teste 1 — O núcleo do ROS 2 está vivo

Terminal 1: `ros2 run demo_nodes_cpp talker`; terminal 2: `ros2 run demo_nodes_py listener`. **Passa:** o *listener* imprime `I heard: [Hello World: 1]`, 2, 3… a condizer com o *talker*. **Falha:** não ouve nada — ROS não carregado, ou `ROS_DOMAIN_ID` diferente nos dois terminais.

Teste 2 — Os pacotes UFACTORY estão visíveis

```
ros2 pkg list | grep -E "xarm|uf_"
```

Passa: pelo menos `uf_ros_lib`, `xarm_api`, `xarm_controller`, `xarm_description`, `xarm_gazebo`, `xarm_moveit_config`, `xarm_msgs`, `xarm_planner`.

Teste 3 — O modelo do robô carrega

```
ros2 launch xarm_moveit_config lite6_moveit_fake.launch.py
```

Passa: o RViz abre com o Lite 6 e um painel **MotionPlanning** (controladores simulados, sem robô envolvido). Deixem a correr para os testes 4 e 5.

Teste 4 — A cadeia cinemática

Num terminal novo:

```
cd ~ && ros2 run tf2_tools view_frames
```

Passa: aparece um `frames_<timestamp>.pdf`. É a cadeia cinemática do Lite 6. Contem os referenciais entre a base e a ponta do braço e **anotem os nomes exatos do primeiro e do último** (provavelmente `link_base` e `link_eef` — confirmem no vosso PDF).

Teste 5 — Ler uma transformação

`ros2 run tf2_ros tf2_echo link_base link_eef` (com os nomes do teste 4). **Passa:** uma translação, uma rotação e uma matriz 4×4 impressas repetidamente. No RViz, arrastem o marcador interativo, cliquem **Plan** e depois **Execute**, e observem os números a mudar. **Registem a matriz 4×4 para duas poses**: **Pose A**, a configuração `home`, e **Pose B**, qualquer outra, cada uma com os ângulos de junta que a produziram.

Teste 6 — O Gazebo corre sozinho

`gz sim shapes.sdf`. **Passa:** uma janela 3D com formas coloridas; carreguem em *play* e o relógio conta. Sejam pacientes no primeiro arranque — descarrega modelos e compila *shaders*. **Janela preta ou crash:** `export LIBGL_ALWAYS_SOFTWARE=1` e repitam; se resultar, acrescentem ao `~/.bashrc`.

Teste 7 — O Lite 6 no Gazebo, sob controlo do MoveIt

```
ros2 launch xarm_moveit_config lite6_moveit_gazebo.launch.py
```

Isto arranca o Gazebo com o Lite 6, o RViz com MotionPlanning, e a ponte `ros2_control` entre eles; deem-lhe até um minuto. **Passa:** (1) o braço fica parado sob a gravidade no Gazebo; (2) Plan + Execute funciona no RViz; (3) **o braço move-se no Gazebo**, não só no RViz. Se o RViz se move e o Gazebo não, os controladores não estão ligados — registem como falha e copiem o primeiro erro que mencione `controller_manager` ou `gz_ros2_control`. Depois observem as duas janelas em conjunto: o braço do Gazebo atrasa-se, ultrapassa e assenta; o do RViz nunca, porque desenha o *plano* enquanto o Gazebo mostra algo mais próximo do *resultado*. **Escrevam uma frase a descrever um comportamento que viram no Gazebo e que o RViz não mostrou.**

Teste 8 — O SDK contra o simulador de firmware

Com o contentor de firmware a correr (§C.4), abram `http://127.0.0.1:18333` e movimentem o braço virtual. Depois corram um script do vosso projeto, a partir do IDE, com o IP vindo do `config/robot.yaml` definido como `127.0.0.1`. **Passa:** ângulos e pose são impressos e o braço move-se na vista do Studio. Chamem `arm.get_forward_kinematics()` para os ângulos da **Pose A** e comparem o resultado com a matriz tf2 do teste 5 — devem concordar em posição bem abaixo do milímetro. **Anotem a diferença.**

Why this matters in practice

Aquela matriz 4×4 do teste 5, e a pose do teste 8, são a mesma grandeza calculada por duas ferramentas independentes. No **LW1** vão calculá-la de uma terceira maneira — a partir da geometria do braço, com código vosso — e verificá-la contra ambas. Guardem os números.

D. Trabalho a Realizar — E0 Parte 2

1. **Nível intermédio**: §B.1–B.4, validado pelo §B.5.
2. **Nível avançado**: Via 1 ou Via 2, validada pelos testes 1–8 do §C.5.
3. **Exercício de ciclo de vida**, usando os comportamentos construídos na parte 1:

(a) Escrevam a **tabela de requisitos** de `safe_session` e `home`: pelo menos 3 funcionais, 2 de desempenho (métrica, limiar, alvo), 1 de interface e 2 de segurança, cada um com um identificador de teste.
(b) **Executem esses testes** contra o simulador de firmware e, onde o docente autorizar, contra o braço; registem passa/falha e valores medidos.
(c) Produzam a **matriz de rastreabilidade**: requisito → módulo → teste → resultado.
(d) Esbocem, numa figura, o **modelo em V desta unidade curricular**: os módulos de LW1–LW4 no ramo esquerdo, os seus testes no direito, o Desafio Final no topo.

4. **Verificação de reprodutibilidade**: a partir de uma *shell* limpa, sigam apenas o vosso `README` e repitam o §B.5 e os testes 1, 3 e 8. Se um passo não for reprodutível a partir das vossas próprias notas, o ambiente não está validado.

Entregue com a Entrega 1 (01/11/2026)

- Uma secção de relatório com **máx. 4 páginas**: o nível e a via escolhidos (básico / intermédio / avançado; Docker ou de raiz; VM ou nativo), o vosso `ROS_DOMAIN_ID`, passa/falha dos testes 1–8 com a *primeira* mensagem de erro em texto para cada falha, as duas matrizes 4×4 e os respetivos ângulos de junta, a frase do teste 7 (Gazebo vs. RViz) e a comparação do teste 8.
- O **exercício de ciclo de vida**: tabela de requisitos, resultados dos testes, matriz de rastreabilidade, esboço do modelo em V (máx. 2 páginas).
- O **URL do repositório** com as suas etiquetas, o `requirements.txt`, o `config/robot.yaml` e o `README`.

Anexo A — Problemas comuns

`ros2: command not found`
O ROS não está carregado neste terminal: `source ~/.bashrc`. Se não resolver, o passo da *shell* do §C.3 não foi concluído.

`Package 'xarm_moveit_config' not found`
Workspace não carregado, ou a compilação falhou: `source ~/dev_ws/install/setup.bash`; se continuar, recompilem e leiam o *primeiro* erro.

`colcon build` falha com *headers* em falta no `xarm_sdk`
Clonado sem `-recursive`; ver a nota no fim do §C.3.

`colcon build` esgota a memória
`colcon build -parallel-workers 2`.

O *listener* não ouve nada no teste 1
Os dois terminais precisam do mesmo `ROS_DOMAIN_ID`: `echo $ROS_DOMAIN_ID` em cada um.

Sem interface gráfica a partir do contentor
Linux: `xhost +local:docker` e as opções `DISPLAY` / `/tmp/.X11-unix`. macOS: XQuartz com "Allow connections from network clients".

O Gazebo abre uma janela preta
`export LIBGL_ALWAYS_SOFTWARE=1`. Numa VM, ativem a aceleração 3D.

O Gazebo fica minutos parado no primeiro arranque
Normal: descarrega modelos e compila *shaders*. Só uma vez.

O braço aparece no Gazebo mas não se move com Execute
O *plugin* `ros2_control` não está a carregar. Verifiquem que `ros-jazzy-gz-ros2-control` está instalado; copiem o primeiro erro que mencione `controller_manager`.

O Gazebo está inutilizavelmente lento
Esperado em VM, em WSL2 e em Docker sem *passthrough* de GPU. Digam-no no relatório; podem usar uma máquina do laboratório para o trabalho com Gazebo.

A compilação funcionava ontem e falha hoje depois de `git pull`
Os submódulos também mudam. Dentro de `xarm_ros2`: `git pull -recurse-submodules`. Depois, em `~/dev_ws`: `rm -rf build install log` e `colcon build`.

SDK: `connection refused` em 127.0.0.1
O firmware não foi arrancado dentro do contentor, ou os portos não foram publicados no `docker run`.

SDK: erro de limite de junta contra o simulador
Instanciem com `check_joint_limit=False` quando falam com o simulador.

Um script corre no terminal mas não no IDE
O IDE está a usar outro interpretador. Apontem-no a `~/venvs/robotica`.

Anexo B — Atribuição de `ROS_DOMAIN_ID`

Grupo	Membros	ID	Grupo	Membros	ID
1		1	9		9
2		2	10		10
3		3	11		11
4		4	12		12
5		5	13		13
6		6	14		14
7		7	15		15
8		8	16		16

Ambos os membros de um grupo usam o mesmo ID. As máquinas da equipa docente usam o ID 0.

[EN] — English Version

Where this fits

In **Part 1** you drove the robot with the tools the robot itself provides. That is the *basic level*, and it does not scale: you cannot version a script typed into a web page, and you cannot test a trajectory that has nowhere to run except the real arm.
This document does two things. First it introduces the **development lifecycle** used for robotic systems — how requirements, design, tests and configuration are managed — which is the discipline every labwork and the Final Challenge report are assessed against. Then it sets up the two remaining working levels:

Level	What you work with	Used for
Basic (Part 1)	Studio, Live Control, the robot's Python IDE	first contact, quick checks
Intermediate	your computer: virtual environment, project, IDE, the SDK	**LW1–LW4 and the Final Challenge**
Advanced	ROS 2 Jazzy + MoveIt 2 + Gazebo	simulation, physics, the vision extension

The Final Challenge is issued in this session, as `Robotics_2627_Challenge`. Read it alongside Part A below: it is a worked example of everything this document teaches about requirements and verification, and it is the target the four remaining labworks build towards. It is demonstrated after the Christmas break, on a date agreed with the class.
Assessment. LW0 is worth 0.4 values, delivered with LW1 and LW2 in **Delivery 1 (01/11/2026)** and assessed at **Checkpoint 1** (Week 8): 75% delivery, 25% class follow-up, plus an individually scored oral defence.

A. The Development Lifecycle for Robotic Systems

A robotic system fails in ways that pure software does not. It has mass and momentum, sensors that lie, a controller with its own opinions, and a person within reach. The lifecycle discipline used in safety-related engineering exists to make those failures visible *before* they happen, and cheaply. This course adopts a light version of it from now on, and assesses it explicitly in the Final Challenge, whose report is a V-model specification.

A.1 Lifecycle models, and why robotics leans on the V

- **Waterfall**: requirements, design, implementation, test, in sequence. Works when nothing is learnt along the way; robotics learns something at every layer.
- **V-model**: every definition level on the left branch is paired with a verification level on the right, and *the test is planned before the code is written.* This is the standard shape in automotive, medical and robotic systems, because testing with hardware is expensive and dangerous, so it must be designed rather than improvised.
- **Iterative / incremental**: short cycles, each delivering something that works and has a stable interface. This course is exactly that: each labwork is an increment with a **frozen interface** — `fk_lite6`, `ik_lite6`, `stream`, `slide_piece`, `pick_and_place` — consumed by the next.

The course runs **increments inside a V**: you climb the left branch labwork by labwork, and the Final Challenge walks the right branch back up.

Lifecycle stage	For the Lite 6 cell	Where
Concept of operations	the task, the cell, the actors, the constraints (reach, payload, stroke)	Challenge
Requirements	functional / performance / interface / safety, each with a test	every sheet
Architecture	modules with frozen interfaces and a data flow	LW1–LW4
Detailed design	DH table, IK branches and policy, damping law, push geometry, blends	LW1–LW4
Implementation	Python modules, `git`, `README`, configuration file	all
Unit verification	FK round trip, analytical vs. numerical Jacobian, feasibility checks	LW1–LW4
Integration verification	a slide and a capture with neighbours occupied	LW4
System verification	the 20-operation campaigns, induced faults, timing	Challenge
Validation	the live session in which the system is confronted with its own specification	Challenge
Operation & maintenance	logs of every motion and return code, post-hoc analysis	Challenge

A.2 Requirements that can be tested

A requirement is one sentence with *shall*, one subject and one measurable outcome. Four kinds: **functional** (what the system does), **performance** (how well, with a metric, a threshold and a target), **interface** (formats, units, protocols — millimetres and degrees in the SDK, radians in ROS 2, a real source of bugs), and **safety** (what must never happen).

Not a requirement	A requirement
The robot shall be accurate.	The system shall place a piece within 5 mm (threshold) / 2 mm (target) of the destination square centre, measured over 20 placements.
The robot shall be safe.	The system shall abort and retreat to `home` within 15 s of detecting that the manipulability has fallen below the configured threshold, verified by three induced-fault tests.
The push shall be smooth.	The tool height shall vary by less than 3 mm peak-to-peak during a slide, reconstructed from the logged joint angles.

Traceability is the thread that holds the V together: requirement → design element → test → measured result. A requirement with no test is a wish; a test that traces to nothing is wasted effort. Every sheet in this course states its requirements as Rn and its performance requirements as Pn with threshold and target, and asks for measured values. The Final Challenge document already contains a worked example of the whole structure: ten functional requirements, ten performance requirements, a fifteen-test protocol, a **coverage matrix** showing that no requirement is left untested, and a traceability table. **Read it alongside this section** — it is the format you will be assessed on, and its validation session is scored on exactly how much of those two lists you complete.

A.3 Verification strategy: the simulation ladder

1. **Unit tests, no robot.** Pure Python: forward-kinematics round trips, finite-difference Jacobians, feasibility checks. Seconds to run; run them on every change.

2. **Simulation in the loop.** Two different simulators, two different purposes (§C.4). Design your code so that switching between simulator and robot is **one parameter**, never an edit.

3. **Hardware in the loop.** Reduced mode, teacher sign-off, a member on the emergency stop. Everything measured, nothing claimed.

4. **Regression.** When LW2 changes something LW1 relied on, the LW1 tests still run. Keep them; they are your safety net in December.

Reporting rule for the whole course: a performance requirement **not met**, with the measured value and an analysis of why, is worth more than one asserted met without evidence.

A.4 Safety inside the lifecycle

Safety is engineered, not hoped for. For this cell the hazards are concrete: pinching at the gripper (6 N is small, fingers are smaller), impact from a 7.2 kg arm at up to 500 mm/s, unexpected motion when an error is cleared, and a shared network on which several laptops can command one controller. The risk-reduction hierarchy as applied here:

Level	In this laboratory
Inherently safe design	reduced-mode speed caps, TCP boundary enclosing the bench, low payload
Technical protective measures	emergency stop, collision detection, the singularity abort you write in LW3
Procedures	teacher sign-off, simulator before hardware, one member on the stop
Information for use	Part 1 §3, the safety declaration you signed

Safety requirements are requirements like any other, with tests like any other. For orientation only, and not examinable: ISO/IEC/IEEE 15288 and 12207 describe system and software lifecycle processes; IEC 61508 is the origin of the V-model discipline in functional safety; ISO 10218 and ISO/TS 15066 cover industrial and collaborative robot safety.

A.5 Configuration and change management

- **One `git` repository per group.** Tag what you deliver: `delivery-1`, `delivery-2`, `delivery-3`. What is assessed is the tag.
- **Parameters are data, not code.** Controller IP, `ROS_DOMAIN_ID`, tool offset, board pose, reduced-mode caps, and later v_{peak}, v_{contact}, h_{push}, h_{fly} live in a configuration file. The Final Challenge forbids code changes during the validation session but allows parameter changes — that rule is only useful to a group that separated the two in October.
- **The `README` reproduces the deliverable from a cold start.** If it cannot, the deliverable does not exist.
- **Logs from day one.** Every commanded motion, the return code, the outcome. The maintenance phase of a robot starts the moment it first moves.

B. Intermediate Level — Your Own Computer

The web Python IDE of Part 1 is replaced from here on. Everything you deliver from LW1 onwards is a Python project on your own machine, under version control, run from a real editor.

B.1 Virtual environment and packages

A virtual environment isolates this course's packages from the rest of your system. Python ≥ 3.10.

```
python3 -m venv ~/venvs/robotica
source ~/venvs/robotica/bin/activate        # Windows: %USERPROFILE%\venvs\robotica\Scripts\activate
python -m pip install --upgrade pip
pip install xarm-python-sdk numpy scipy matplotlib pytest pyyaml
pip freeze > requirements.txt               # commit this file
```

B.2 The repository and the SDK

```
git clone https://github.com/xArm-Developer/xArm-Python-SDK.git   # reference: doc/api, example/wrapper
```

Clone it **outside** your project: it is documentation and examples, not your code. The package you import comes from `pip`, not from this clone.

B.3 Project structure

Use this layout from the first day. It is the structure the later labworks assume when they say "import your LW1 module".

```
robotica-grupoNN/
  config/robot.yaml          # ip, home, tool offset, speed caps, board pose, v_peak, v_contact...
  lite6/
    __init__.py
    session.py               # safe_session() from Part 1
    kinematics.py            # LW1: fk_lite6, pose_to_T, T_to_pose, workspace_sample
    ik.py                    # LW2: ik_lite6
    velocity.py              # LW3: jacobian, manipulability, rrmc_step, stream
    trajectory.py            # LW4: traj_*, slide_piece, pick_and_place
    game.py                  # Final Challenge: board state, rules, turn sequencer
  scripts/                   # one runnable script per experiment, named after the test it runs
  tests/                     # pytest: unit tests that need no robot
  logs/
  requirements.txt
  README.md
```

Why this matters in practice

Why a package and not a folder of scripts. Each labwork freezes an interface that the next imports: `from lite6.kinematics import fk_lite6`. If LW1 lives in `trabalho1_final_v3.py`, LW2 cannot import it without copying, and by December you will have four copies of a forward-kinematics function that disagree with each other. This is the configuration-management point of §A.5, in its most concrete form.

B.4 The IDE

Use **VS Code** (with the Python and Pylance extensions) or **PyCharm**. Either is fine; what matters is that you get: the interpreter pointed at your virtual environment, a debugger with breakpoints, and integrated `git`. Set the interpreter explicitly — a script that runs in the terminal but not in the IDE is almost always a wrong interpreter.

B.5 Validation of the intermediate level

1. `pytest` runs and passes on an empty test suite.
2. `safe_session.py` from Part 1, moved into `lite6/session.py`, is imported by a script in `scripts/` and connects to the robot.
3. Every parameter the script needs comes from `config/robot.yaml`; the word `192.168` appears nowhere in the Python source.
4. The repository has at least one commit and the `README` explains how to recreate the environment.

C. Advanced Level — ROS 2 and Gazebo

C.1 Why

The SDK talks to the controller; it does not simulate a robot. **Gazebo** does: mass, gravity, inertia, joint friction, contact. An arm in Gazebo sags under its own weight, overshoots and does not track its commanded trajectory perfectly — like the real one. **RViz** is a viewer, not a simulator: it draws the robot where the kinematics say it should be and is always right, because it is not simulating anything. **MoveIt 2** plans collision-free motions and gives you an independent reference to check your own planner against. Distinguishing these three is half the value of this section.

The target stack is **ROS 2 Jazzy Jalisco** + MoveIt 2 + Gazebo Harmonic + the UFACTORY `xarm_ros2` packages, on **Ubuntu 24.04**. Two routes: the container (recommended) and the from-scratch install (optional).

C.2 Route 1 — Docker (recommended)

A container gives every group the same environment regardless of what their laptop runs, and removes an afternoon of package management from the critical path.

```
# image supplied by the teaching team
docker pull <registry>/robotica-deec/ros2-jazzy-lite6:2627

# Linux: GUI applications from the container on your display
xhost +local:docker
docker run -it --name ros2lab --net=host \
    -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix \
    -v $HOME/robotica-grupoNN:/work \
    <registry>/robotica-deec/ros2-jazzy-lite6:2627
```

On **Windows** use Docker Desktop with the WSL2 backend; on **macOS** use Docker Desktop and an X server (XQuartz) for the GUI, and expect Gazebo to be slow on Apple Silicon, where the image is emulated. Mount your project directory (`-v` above) so that the code you edit in your IDE on the host is the code that runs in the container.

C.3 Route 2 — From scratch (optional, for the curious)

Worth doing if you want to understand what the container contains. Two variants: a **virtual machine** (VMware Workstation on Windows/Linux, VMware Fusion on macOS; give it at least 4 cores, 8 GB RAM and 40 GB disk, and enable 3D acceleration or Gazebo will not render) or a **native** Ubuntu 24.04 install, which is the fastest of all.

1. **Locale.**

```
sudo apt update && sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

2. **Repositories.**

```
sudo apt install -y software-properties-common
sudo add-apt-repository universe
```

3. **ROS 2 apt source.** Follow the *Set up sources* section of the official page, `https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html`. This is the one step upstream changes from time to time, so the official page is the source of truth. Stop when `sudo apt update` runs cleanly.

4. **ROS 2, tools, MoveIt, Gazebo.** ROS 2 Jazzy pairs with **Gazebo Harmonic**; do not install "Gazebo Classic", which reached end of life in January 2025 and is a different, incompatible program despite the name.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ros-jazzy-desktop ros-dev-tools ros-jazzy-moveit \
                    ros-jazzy-ros-gz ros-jazzy-gz-ros2-control
sudo rosdep init && rosdep update
gz sim --version        # expect 8.x  (Gazebo Sim 8 is Harmonic)
```

5. **Shell.** Add to `~/.bashrc`, then `source` it:

```
source /opt/ros/jazzy/setup.bash
export ROS_DOMAIN_ID=<YOUR GROUP NUMBER>
```

   `ROS_DOMAIN_ID` **is not optional.** ROS 2 nodes discover each other over the local network. Without it, on the lab LAN, your RViz will find your classmates' nodes, you will see joint states that are not yours, and MoveIt will plan against a robot that does not exist. Use the number assigned to your group (Appendix B) and no other.

6. **Workspace and `xarm_ros2`.** Order matters: `$ROS_DISTRO` must already be set, which is why the previous step comes first.

```
mkdir -p ~/dev_ws/src && cd ~/dev_ws/src
# DO NOT omit --recursive: the C++ SDK that talks to the robot is a git submodule
git clone https://github.com/xArm-Developer/xarm_ros2.git --recursive -b jazzy
rosdep install --from-paths . --ignore-src --rosdistro jazzy -y
cd ~/dev_ws && colcon build        # 15-30 min; --parallel-workers 2 if RAM is
    short
echo "source ~/dev_ws/install/setup.bash" >> ~/.bashrc && source ~/.bashrc
```

   Forgot `-recursive`? No need to re-clone: in `~/dev_ws/src/xarm_ros2` run `git submodule update -init -recursive`, then rebuild.

C.4 Two simulators, two purposes

	Gazebo (ROS 2)	**UFACTORY firmware simulator**
What it reproduces	the *physics*: mass, gravity, friction, contact	the *controller*: modes, servo streaming, limits, error codes
Driven by	MoveIt 2 through `ros2_control`	the xArm Python SDK, exactly as the real arm is
Use it before	any MoveIt-planned motion	any SDK-driven motion, which is all of LW3 and LW4

The firmware simulator is UFACTORY's own controller firmware distributed as a container. It speaks the same protocol as the physical arm, so **the same Python script runs unchanged against it and against the robot** — only the IP changes, which is why the IP belongs in `config/robot.yaml`. It is amd64 only, and emulated on Apple Silicon.

```
docker pull danielwang123321/uf-ubuntu-docker
docker run -it --name uf_software -p 18333:18333 -p 502:502 -p 503:503 -p 504:504 \
    -p 30000:30000 -p 30001:30001 -p 30002:30002 -p 30003:30003 \
    danielwang123321/uf-ubuntu-docker
```

```
# inside the container, start the firmware selecting the Lite 6 profile (arguments "6
    9");
# see the UFACTORY support article "How to install UFACTORY Studio in Docker"
# then: browser -> http://127.0.0.1:18333   Python -> XArmAPI('127.0.0.1',
    check_joint_limit=False)
```

C.5 Testing the advanced level

Run these **in order**; each assumes the previous passed. Open a new terminal for each unless told otherwise. Record every result — this is a deliverable.

Test 1 — ROS 2 core is alive

Terminal 1: `ros2 run demo_nodes_cpp talker`; terminal 2: `ros2 run demo_nodes_py listener`. **Pass:** the listener prints `I heard: [Hello World: 1]`, 2, 3... matching the talker. **Fail:** nothing heard — ROS not sourced, or a different `ROS_DOMAIN_ID` in the two terminals.

Test 2 — The UFACTORY packages are visible

```
ros2 pkg list | grep -E "xarm|uf_"
```

Pass: at least `uf_ros_lib`, `xarm_api`, `xarm_controller`, `xarm_description`, `xarm_gazebo`, `xarm_moveit_config`, `xarm_msgs`, `xarm_planner`.

Test 3 — The robot model loads

```
ros2 launch xarm_moveit_config lite6_moveit_fake.launch.py
```

Pass: RViz opens with the Lite 6 and a **MotionPlanning** panel (simulated controllers, no robot involved). Leave it running for Tests 4 and 5.

Test 4 — The kinematic chain

In a new terminal:

```
cd ~ && ros2 run tf2_tools view_frames
```

Pass: a `frames_<timestamp>.pdf` appears. It is the kinematic chain of the Lite 6. Count the frames between the base and the end of the arm, and **write down the exact names of the first and last** (probably `link_base` and `link_eef` — confirm from your own PDF).

Test 5 — Read a transform

`ros2 run tf2_ros tf2_echo link_base link_eef` (substitute the names from Test 4). **Pass:** a translation, a rotation and a 4×4 matrix printed repeatedly. In RViz, drag the interactive marker, click **Plan** then **Execute**, and watch the numbers change. **Record the 4×4 matrix for two poses**: **Pose A**, the home configuration, and **Pose B**, any other, each with the joint angles that produced it.

Test 6 — Gazebo runs on its own

`gz sim shapes.sdf`. **Pass:** a 3D window with coloured shapes; press play and the clock counts. Be patient on the first launch — it downloads models and compiles shaders. **Black window or crash:** `export LIBGL_ALWAYS_SOFTWARE=1` and retry; if that works, add it to `~/.bashrc`.

Test 7 — The Lite 6 in Gazebo, under MoveIt control

```
ros2 launch xarm_moveit_config lite6_moveit_gazebo.launch.py
```

This starts Gazebo with the Lite 6, RViz with MotionPlanning, and the `ros2_control` bridge between them; give it up to a minute. **Pass:** (1) the arm stands still under gravity in Gazebo; (2) Plan + Execute works in RViz; (3) **the arm moves in Gazebo**, not only in RViz. If RViz moves and Gazebo does not, the controllers are not connected — record it as a failure and copy the first error mentioning `controller_manager` or `gz_ros2_control`. Then watch both windows together: the Gazebo arm lags, overshoots and settles; RViz never does, because it draws the *plan* while Gazebo shows something closer to the *outcome*. **Write one sentence describing a behaviour you saw in Gazebo that RViz did not show.**

Test 8 — The SDK against the firmware simulator
With the firmware container running (§C.4), open `http://127.0.0.1:18333` and jog the virtual arm. Then run a script from your project, from your IDE, with the IP taken from `config/robot.yaml` set to `127.0.0.1`. **Pass:** angles and pose are printed and the arm moves in the Studio view. Call `arm.get_forward_kinematics()` for the joint angles of **Pose A** and compare the result with the tf2 matrix of Test 5 — they should agree in position to well under a millimetre. **Write down the difference.**

Why this matters in practice

That 4×4 matrix from Test 5, and the pose from Test 8, are the same quantity computed by two independent tools. In **LW1** you will compute it a third way — from the geometry of the arm, with code you wrote — and check it against both. Keep the numbers.

D. Work to be Performed — E0 Part 2

1. **Intermediate level**: §B.1–B.4, validated by §B.5.
2. **Advanced level**: Route 1 or Route 2, validated by Tests 1–8 of §C.5.
3. **Lifecycle exercise**, using the behaviours you built in Part 1:
   (a) Write the **requirements table** for `safe_session` and `home`: at least 3 functional, 2 performance (metric, threshold, target), 1 interface and 2 safety requirements, each with a test identifier.
   (b) **Execute those tests** against the firmware simulator and, where the teacher authorises, the arm; record pass/fail and measured values.
   (c) Produce the **traceability matrix**: requirement → module → test → result.
   (d) Sketch, in one figure, the **V-model of this course**: LW1–LW4 modules on the left branch, their tests on the right, the Final Challenge at the top.
4. **Reproducibility check**: from a clean shell, follow only your own `README` and repeat §B.5 and Tests 1, 3 and 8. If a step cannot be reproduced from your own notes, the environment is not validated.

Submitted with Delivery 1 (01/11/2026)

- A report section of **max. 4 pages**: the level and route chosen (basic / intermediate / advanced; Docker or from scratch; VM or native), your `ROS_DOMAIN_ID`, pass/fail for Tests 1–8 with the *first* error message as text for any failure, the two 4×4 matrices and their joint angles, the Test 7 Gazebo-vs-RViz sentence, and the Test 8 comparison.
- The **lifecycle exercise**: requirements table, test results, traceability matrix, V-model sketch (max. 2 pages).
- The **repository URL** with its tags, `requirements.txt`, `config/robot.yaml` and the `README`.

Appendix A — Common problems

`ros2: command not found`
ROS is not sourced in this terminal: `source ~/.bashrc`. If that does not help, the shell step of §C.3 was not completed.

`Package 'xarm_moveit_config' not found`
Workspace not sourced, or the build failed: `source ~/dev_ws/install/setup.bash`; if it still fails, rebuild and read the *first* error.

`colcon build` fails with missing headers in `xarm_sdk`
Cloned without `-recursive`; see the note at the end of §C.3.

`colcon build` exhausts memory
`colcon build -parallel-workers 2`.

The listener hears nothing in Test 1
Both terminals need the same `ROS_DOMAIN_ID`: `echo $ROS_DOMAIN_ID` in each.

No GUI from the container
Linux: `xhost +local:docker` and the `DISPLAY` / `/tmp/.X11-unix` options. macOS: XQuartz with "Allow connections from network clients".

Gazebo opens a black window
`export LIBGL_ALWAYS_SOFTWARE=1`. In a VM, enable 3D acceleration.

Gazebo hangs for minutes on first launch
Normal: it downloads models and compiles shaders. Once only.

The arm appears in Gazebo but does not move on Execute
The `ros2_control` plugin is not loading. Check `ros-jazzy-gz-ros2-control` is installed; copy the first error naming `controller_manager`.

Gazebo is unusably slow
Expected in a VM, in WSL2 and in Docker without GPU passthrough. Say so in the report; you can use a laboratory machine for the Gazebo work.

The build worked yesterday and fails today after `git pull`
Submodules move too. Inside `xarm_ros2`: `git pull -recurse-submodules`. Then, in `~/dev_ws`: `rm -rf build install log` and `colcon build`.

SDK: `connection refused` on 127.0.0.1
The firmware was not started inside the container, or the ports were not published in `docker run`.

SDK: joint limit error against the simulator
Instantiate with `check_joint_limit=False` when talking to the simulator.

A script runs in the terminal but not in the IDE
The IDE is using a different interpreter. Point it at `~/venvs/robotica`.

Appendix B — `ROS_DOMAIN_ID` allocation

Group	Members	ID	Group	Members	ID
1		1	9		9
2		2	10		10
3		3	11		11
4		4	12		12
5		5	13		13
6		6	14		14
7		7	15		15
8		8	16		16

Both members of a group use the same ID. The teaching team's machines use ID 0.